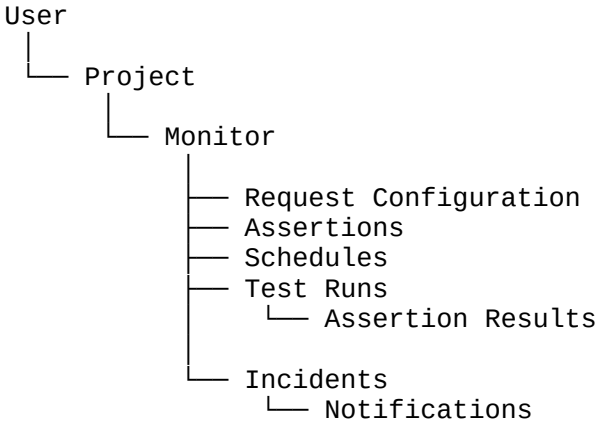
Domain Model

The core relationship should be:



1. User

Represents the person using the platform.

Core data:

- id
- name
- email
- password hash
- createdAt
- updatedAt

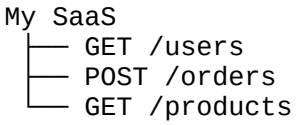
For MVP, one user can own multiple projects.

---

2. Project

Groups related API monitors.

Example:



Core data:

- id
- userId
- name
- description
- createdAt
- updatedAt

Relationship:

```
User 1 ——— * Projects
```

---

3. Monitor

This is the **central entity**.

A monitor represents one continuously tested API endpoint.

Example:

```
Name: Create Order
Method: POST
URL: https://api.example.com/orders
```

Core data:

- id
- projectId
- name
- method
- url
- timeout
- enabled
- createdAt
- updatedAt

Relationship:

```
Project 1 ——— * Monitors
```

---

4. Request Configuration

This needs some thought.

A monitor needs:

- Headers
- Query parameters
- Request body
- Authentication

I would **not immediately create separate tables for every one**.

For MVP, PostgreSQL `JSONB` is perfectly reasonable.

Example:

```
{
  "Content-Type": "application/json",
  "Accept": "application/json"
```

```
}
```

And:

```
{
  "product_id": 123,
  "quantity": 2
}
```

This keeps the initial schema simpler.

---

5. Authentication

Authentication should be represented separately from arbitrary request headers because credentials are sensitive.

Possible types:

```
NONE
API_KEY
BEARER
BASIC
```

The actual credentials should be encrypted.

We should avoid storing raw API keys in ordinary JSON configuration.

---

6. Assertion

An assertion defines what the API **should do**.

Example:

```
Status equals 200
Response time < 500ms
$.data.id exists
$.data.email type = string
$.data.status equals "active"
```

Core data:

- id
- monitorId
- type
- target/path
- operator
- expectedValue
- createdAt
- updatedAt

Relationship:

```
Monitor 1 ——— * Assertions
```

---

7. Schedule

Defines how frequently a monitor runs.

Example:

```
Every 5 minutes
```

I'd keep the schedule attached to the monitor initially.

Potential fields:

- enabled
- interval
- nextRunAt

We don't necessarily need a separate `Schedule` table in MVP.

---

8. Test Run

This represents **one execution** of a monitor.

Example:

```
Monitor:
GET /users

Executed:
14:05:32

Result:
FAILED

Status:
500

Response:
1,842ms
```

Core data:

- id
- monitorId
- status
- startedAt
- completedAt
- duration
- httpStatus
- error
- response metadata

Relationship:

```
Monitor 1 ——— * TestRuns
```

This will become one of our largest tables.

---

9. Assertion Result

An assertion defines what should happen.

An assertion result records what actually happened during a specific test.

Example:

```
Assertion:
Status = 200

Result:
Expected: 200
Actual: 500
FAILED
```

Relationship:

```
TestRun 1 ——— * AssertionResults
Assertion 1 ——— * AssertionResults
```

This distinction is important.

Don't put the result directly on the `Assertion` because the same assertion will run thousands of times.

---

10. Incident

An incident represents a **continuous period of failure**, not an individual failed test.

Example:

```
Incident #124

Monitor:
POST /orders

Started:
14:02

Resolved:
14:18

Duration:
16 minutes

Failed runs:
16
```

Core data:

- id
- monitorId
- status
- startedAt
- resolvedAt
- failureReason
- createdAt
- updatedAt

Relationship:

```
Monitor 1 ——— * Incidents
```

---

11. Notification

Represents an alert generated by an incident.

Example:

```
Incident
  ↓
Email notification
  ↓
Delivered
```

Eventually:

```
Incident
  ├── Email
  ├── Webhook
  └── Slack
```

For MVP:

```
EMAIL
WEBHOOK
```

---

12. Notification Configuration

This is different from a notification event.

Configuration

> "Send failures to this email."

Notification

> "Failure notification for Incident #124 was sent."

So eventually we'll have:

```
NotificationConfig
        ↓
Notification
        ↓
Incident
```

---

13. API Key / Credentials

This deserves special treatment.

The platform itself may eventually expose an API for developers.

That means we could have:

```
User
  ↓
API Key
```

But I would **not build public API keys into MVP unless we actually need them**.

Don't create infrastructure just because we might need it later.

---

14. Final MVP Entity Map

I'd keep the first database around these entities:

```
User
 │
 └── Project
       │
       └── Monitor
             │
             ├── Authentication
             ├── Assertions
             ├── TestRuns
             │      │
             │      └── AssertionResults
             │
             └── Incidents
                    │
                    └── Notifications
```